# Dependency Injection

Session 1: Fundamentals of Dependency Injection

# What Is Dependency Injection

- Giving a class (client) the objects (services) it needs from outside.

```
class Engine

class Car(private val engine: Engine)
```

# Why Use DI

- Reduce coupling
- Improve testability
- Better reusability

# Reduce Coupling (No DI)

```
class ApiService {
    fun getUsers(): List<String> = listOf("Alice", "Bob")
}

class UserRepository {
    private val api = ApiService()
    fun fetchUsers() = api.getUsers()
}

class UserViewModel {
    private val repo = UserRepository()
    fun loadUsers() = repo.fetchUsers()
}
```

# Reduce Coupling (DI)

```
class ApiService {
    fun getUsers(): List<String> = listOf("Alice", "Bob")
}

class UserRepository(private val api: ApiService) {
    fun fetchUsers() = api.getUsers()
}

class UserViewModel(private val repo: UserRepository) {
    fun loadUsers() = repo.fetchUsers()
}
```

# Improve Testability

```
class UserRepositoryTest {

    @Test
    fun testFetchUsers() {
        val fakeApi = FakeApiService()
        val repo = UserRepository(fakeApi)

        val result = repo.fetchUsers()

        assert(result.contains("Fake User"))
    }
}
```

# Better Reusability

```
class OrderRepository(private val api: ApiService)
class ProductRepository(private val api: ApiService)
```

# Problem With Manual DI

- Lot's of boilerplate
- Harder to maintain as codebase grows

# Hilt As Dependency Framework

- Reduced boilerplate.
- Intuitive once understood.
- Advance mechanism and capabilities.

# Ways of Injection

- Constructor Injection
- Field Injection
- Method Injection

# Constructor Injection

- Use when you control the class and can inject dependencies via constructor (Most preferred)

# Constructor Injection

```
class UserRepository @Inject constructor(
    private val apiService: ApiService
) {
    fun getUser() = apiService.getUser()
}
```

# Field Injection

- Used when Android creates the class (Activity, Fragment, etc.)

# Field Injection

```
@AndroidEntryPoint
class MainActivity : AppCompatActivity() {

    @Inject lateinit var repository: UserRepository

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)

        repository.getUser()
    }
}
```

# Method Injection

- Used when dependency is needed only at a specific time or setup step.
- When field injection is not possible until certain point of time.

# Method Injection

```
class AnalyticsManager @Inject constructor() {

    private lateinit var logger: Logger

    @Inject
    fun init(logger: Logger) {
        this.logger = logger
    }

    fun track() {
        logger.log("Tracking event")
    }
}
```

# Quick Question

- Does below code snippet violets DI pattern or not?

```
@HiltAndroidApp
class BaseApplication : Application() {

    val list = mutableListOf<String>("A","B","C")

    override fun onCreate() {
        super.onCreate()
    }
}
```

# Hilt Components

- Each dependency is bound to a certain component.
- Components define lifecycle and scope of dependencies.
- Hilt comes with built in components where we can install our dependencies as per our needs.
- We can define our own component if needed.
- Important Components: SingletonComponent, ViewModelComponent, ActivityComponent.

# SingletonComponent

- Lives as long as the application lives.
- Dependencies of this component are accessible across application.

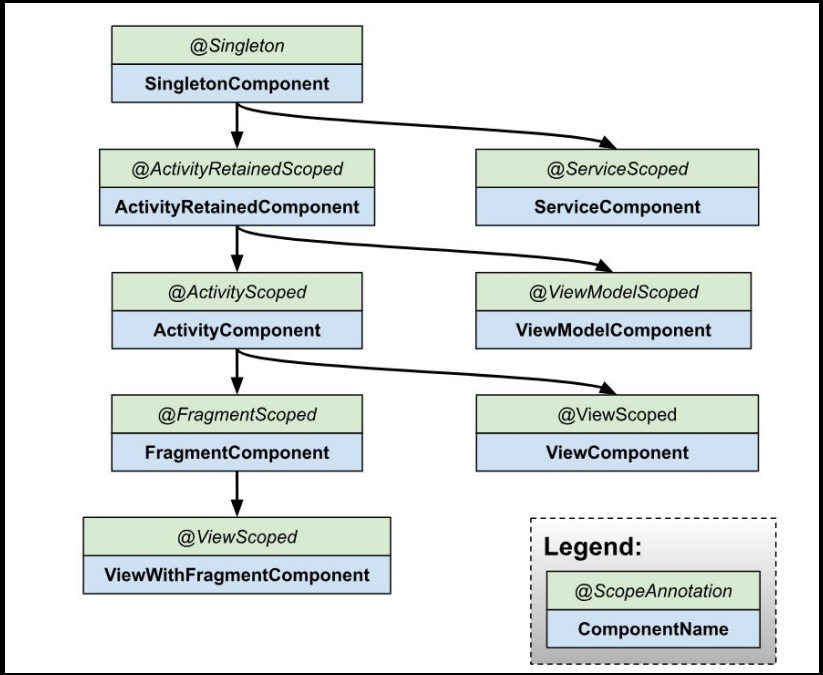
# ViewModel Component

- Lifecycle tied per viewModel.
- Dependencies of this component are accessible across viewModels.

# Activity Component

- Lives per Activity lifecycle.
- Used for activity specific dependencies.

# Component Hierarchy



# Component Default Binding

| Component | Injector for |
| --- | --- |
| SingletonComponent | Application |
| ViewModelComponent | ViewModel |
| ActivityComponent | Activity |
| FragmentComponent | Fragment |
| ViewComponent | View |
| ViewWithFragmentComponent | View with @WithFragmentBindings |
| ServiceComponent | Service |

# Modules

- We use it to create dependencies.
- Also sets in which component these dependencies would be installed/available.

# Modules

```
@Module
@InstallIn( ...value = ActivityComponent::class)
class SnackbarManagerModule {

    1 Usage
    @Provides
    fun provideSnackbarManager(activity: Activity): WeatherSnackbarManager {
        return WeatherSnackbarManager(activity as ComponentActivity)
    }

    1 Usage
    @Provides
    fun provideSnackbarHostState(): SnackbarHostState {
        return SnackbarHostState()
    }
}
```

# Modules

- All modules installed in the same component share their dependencies.

# Modules

```
@Module
@InstallIn(SingletonComponent::class)
object NetworkModule {

    @Provides
    fun provideBaseUrl(): String = "https://api.com"
}
```

```
@Module
@InstallIn(SingletonComponent::class)
object ApiModule {

    @Provides
    fun provideApiService(baseUrl: String): ApiService {
        return ApiService(baseUrl)
    }
}
```